

LEO Ağlarında Gelişmiş TCP Flooding Saldırılarının Tespiti için SDN Temelli Bir Yaklaşım

1st Ebrar ŞAHİN
Uludağ Üniversitesi
İstanbul, Türkiye
ebrarsahin00@gmail.com

Abstract—Bilgisayar Ağlarında Temel Protokoller dersi kapsamında verilen proje ödevinin durum ilerleme raporudur. 7 hafta boyunca çalışma kapsamında neler yapıldığı, elde edilen bulgular, karşılaşılan zorluklar paylaşılmıştır.

Index Terms—LEO, TCP, SDN, Flooding Attacks

I. GENEL ÖZET VE İLERLEME DURUMU

Bu proje, Low Earth Orbit (LEO) uydusu haberleşme ağlarında gerçekleştirilen TCP tabanlı flooding saldırılarının (SYN Flood, ACK Flood) tespit edilmesi ve engellenmesi amacıyla Software Defined Networking (SDN) mimarisine dayalı bir güvenlik sistemi geliştirmeyi hedeflemektedir. Proje kapsamında, LEO uydusu ağ topolojisinin simülasyonu, merkezi SDN kontrolcü tasarımı ve gerçek zamanlı trafik izleme altyapısının oluşturulması planlanmaktadır. Proje, planlanan takvime uygun olarak ilerlemekte olup, temel altyapı bileşenleri %70 oranında tamamlanmıştır. LEO uydusu ağ simülasyon ortamı başarıyla kurulmuş ve çalışır duruma getirilmiştir. Sistem, 3 orbital düzlemde konumlandırılmış 12 LEO uydusu, 4 yer istasyonu ve 12 kullanıcı terminalinden oluşan bir ağ topolojisini Mininet emülatörü üzerinde simüle etmektedir. Ryu tabanlı SDN kontrolcü, OpenFlow 1.3 protokolü aracılığıyla tüm switch'leri merkezi olarak yönetmekte ve 5 saniyelik aralıklarla akış istatistiklerini toplamaktadır. Gerçekleştirilen ping testlerinde, STP yakınsaması sonrasında %96 başarı oranıyla uçtan uca bağlantı sağlanmıştır. Temel altyapının tamamlanmasıyla birlikte, bir sonraki aşamada TCP flooding saldırı simülasyonu ve makine öğrenmesi tabanlı saldırı tespit modülünün geliştirilmesine odaklanılacaktır.

II. PROJE HEDEFLERİNİN VE KAPSAMININ GÖZDEN GEÇİRİLMESİ

Projenin hedefleri Tablo 1'de listelenmiştir. Ağ modelleme aşamasında ilk denemelerde, 66 uydulu tam ölçekli Starlink topolojisinin Mininet üzerinde simüle edilmesinin aşırı bellek tüketimi ve CPU yükü oluşturduğu gözlemlenmiştir. Mininet'in mimari yapısı ve literatürdeki kullanım örnekleri, tam ölçekli büyük topolojilerin (66 uydusu/switch) tek bir makine üzerinde simüle edilmesinin zorluklarını ortaya koymaktadır ve PoC çalışmaları için ölçeğin küçültülmesinin olağan olduğunun vurgusu yapılmıştır. Mininet dokümantasyonunda, 9 switch ve 64 host içeren küçük bir ağ topolojisinin bile bir dizüstü bilgisayarda başlatılmasının yaklaşık 45 saniye sürdüğü belirtilmiştir.

Literatürde küçük ölçekli topoloji kullanan kaynaklardaki örnek senaryolar, genellikle az sayıda switch kullanıldığını göstermektedir. Bu sebeple, Mininet'in tek bir makine üzerindeki kaynaklara bağımlı olması ve literatürdeki benzer çalışmaların karmaşıklığı yönetmek adına genellikle 10'un altında veya o sayıya yakın switch ile çalışması göz önünde bulundurularak [1]; 12 uydulu (switchli) bir sisteme geçiş yapılması proje için uygun görülmüştür.

Projenin bağımlılık yönetiminin kolaylaştırılması amacıyla Docker konteyner mimarisi kullanılmıştır. İlk etapta çeşitli bağımlılık problemleri ile karşılaşınca değişikliğe gidilmiştir. Bu değişiklik, özellikle Mininet'in Python 3 uyumluluğu ve Ryu bağımlılık çakışmalarının yönetilmesinde önemli kolaylık sağlamıştır.

Docker sanallaştırma katmanı ve Mininet'in yazılım tabanlı switch emülasyonu nedeniyle, gerçek zamanlı tepki süresi hedefi esnetilmiştir. Ancak bu sınırlama, sistemin gerçek donanım üzerinde çalıştırılması durumunda aşılabilecek bir simülasyon kısıtı olduğu düşünülmektedir.

TABLE I
PROJE HEDEFLERİ

No	Hedef	Açıklama
1	LEO Ağ Modelleme	LEO uydusu takım yıldızı SDN tabanlı simülasyonu
2	SDN Kontrolcü	Merkezi trafik yönetimi ve izleme altyapısı
3	Saldırı Tespiti	TCP SYN/ACK flooding saldırılarının gerçek zamanlı tespiti
4	Otomatik Müdahale	Tespit edilen saldırılara karşı dinamik akış kuralları ile engelleme
5	Performans Analizi	Sistemin tespit doğruluğu ve tepki süresinin değerlendirilmesi

III. TAMAMLANAN ÇALIŞMALAR

Bu bölümde, projenin başlangıcından itibaren tamamlanan teknik adımlar, karşılaşılan sorunlar ve çözümleri detaylı olarak açıklanmaktadır.

A. Temel Gereksinimler

Aşağıdaki yazılımlar Docker konteyner içerisine kurulmuştur:

- 1) Ubuntu 20.04 LTS: Python 3.8 native desteği, Ryu uyumluluğu
- 2) Python 3.8: Ryu ve eventlet uyumluluğu

- 3) Mininet 2.3: Python 3 desteği, OVS entegrasyonu
- 4) Open vSwitch 2.13: OpenFlow 1.3
- 5) Ryu 4.34: Son sürüm
- 6) eventlet 0.30.2: Ryu ile uyumlu sürüm

Araçların kullanım senaryoları aşağıda kısaca açıklanmıştır:

- 1) Mininet 2.3: Sanal ağ emülatörü. Gerçek Linux kernel, switch ve uygulama kodlarını kullanarak sanal hostlar, switch'ler ve bağlantılar oluşturur. Projede, LEO uydu topolojisini (12 switch, 16 host) simüle etmek için kullanıldı.
- 2) Open vSwitch 2.13: Yazılım tabanlı sanal switch. Projede her uydu bir OVS switch olarak modellendi, akış kuralları bu switch'lere yüklendi.
- 3) Ryu 4.34: Python tabanlı SDN controller framework'ü. OpenFlow mesajlarını işler, switch'leri yönetir.
- 4) eventlet 0.30.2: Python için asenkron I/O kütüphanesi. Ryu'nun dahili olarak kullandığı concurrency mekanizmasıdır. Projede Ryu'nun birden fazla switch ile eşzamanlı iletişim kurmasını sağlar.

İlk geliştirme denemelerinde, Ubuntu 22.04 (Python 3.10) kullanıldığında aşağıdaki hata ile karşılaşılmıştır:

- TypeError: cannot set 'is timeout' attribute of immutable type 'TimeoutError'

Yapılan araştırmalar sonucu Python 3.10'da 'TimeoutError' sınıfının immutable hale getirildiği gözlemlenmiştir. eventlet kütüphanesi de "TimeoutError" sınıfını kullandığından Ryu çalıştırılamamıştır. Ryu'nunda geliştirilmesi durdurulduğundan Python sürümü 3.8'e düşürülmüştür.

B. IP Adresleme

Subnet: 10.0.0.0/24

TABLE II
YER İSTASYONLARI

Host	IP Adresi
gs1	10.0.0.1/24
gs2	10.0.0.2/24
gs3	10.0.0.3/24
gs4	10.0.0.4/24

TABLE III
KULLANICI TERMINALLERİ

Host	IP Adresi
u1	10.0.0.5/24
u2	10.0.0.6/24
u3	10.0.0.7/24
u4	10.0.0.8/24
u5	10.0.0.9/24
u6	10.0.0.10/24
u7	10.0.0.11/24
u8	10.0.0.12/24
u9	10.0.0.13/24
u10	10.0.0.14/24
u11	10.0.0.15/24
u12	10.0.0.16/24

C. Spanning Tree Protokol Kullanımı(STP)

Mininet'in kendi default topolojisinde ping sıkıntısı yaşanmazken PoC için kurulan topolojide pingler başarısız olurken paketlerin droplanıldığı gözlemlendi. LEO topolojisinde ISL (Inter-Satellite Links) bağlantıları mesh/ring yapısı oluşturur. Böyle bir durumda broadcast paketi (ör: ARP) gönderildiğinde paket tüm portlardan çıkar. Komşu switch'ler tekrar tüm portlardan paketi iletir. Paketler sonsuza kadar döner. Bu duruma "broadcast storm" denir. PoC'de broadcast storm sorununa çözüm olarak STP kullanılmıştır. STP ile bazı portlar "blocking" moduna alınır. Döngü kırılarak spanning tree (ağaç) yapısı oluşturulur. Broadcast paketleri sadece ağaç dallarında ilerler. STP öncesi tüm paketler %100 droplanırken STP ile droplanan paketlerin %4 olduğu gözlemlenmiştir. STP sonrası droplanan paketlerin durumu ise STP'nin aktivasyon süresi içerisinde atılan paketler olabileceği düşünülmektedir.

D. SDN

LEO uyduları sürekli hareket halinde olduğundan ağ topolojisi çok hızlı değişmektedir. Geleneksel ağ yapıları, hızlı topoloji değişikliklerinde bağlantı kopmaları ve dengesiz ağ trafiği sorunları yaşamakta olduğu bilinmektedir. SDN, kontrol düzlemini (karar verici) veri düzleminde (iletim yapan cihaz) ayırmaktadır. [2] Böylelikle, ağdaki uyduların her topoloji değişiminde karmaşık rota hesaplamaları yapmasına gerek kalmamaktadır. Merkezi SDN kontrolcüsü, değişen topolojiyi anlık olarak görmekte ve gerekli akış (flow) kurallarını düğümlere göndermektedir. [2] Sonuç olarak bağlantı kopmalarını ve yeniden rota hesaplama sürelerini minimize etmektedir. Binlerce uydudan oluşan bir ağ yapısında, trafiğin en verimli yoldan akması için tüm ağın durumunun bilinmesi gerekmektedir. SDN kontrolcüsü, ağdaki darboğazları veya kopuklukları görerek trafiği dinamik olarak yönlendirmektedir. Bu yapı Google'ın veri merkezlerinde elastik yükleri yönetmek için OpenFlow kullanmasına benzer bir verimlilik sağlamaktadır. [3] Geleneksel dağıtık protokollerin (OSPF, BGP vb.) hızlı değişen ve hareketli uydu topolojilerine adapte olmakta yavaş kalması, buna karşılık SDN'in merkezi ve programlanabilir bir yapıya sahip olması, dinamikliği yönetmekte esnek olması bu proje için onu temel kılmıştır.

Çeşitli SDN frameworkleri arasından bu proje için RYU seçilmiştir. Yaygın olarak kullanılan frameworkler aşağıdaki gibidir:

- 1) RYU(Python)
- 2) OpenDaylight(Java)
- 3) ONOS(Java)
- 4) Floodlight(Java)
- 5) POX(Python)

RYU, Python dili ile geliştirilmiştir. Mininet ağ emülatörünün de topoloji oluşturmak için Python API'lerini kullanması göz önüne alındığında, RYU'nun Python tabanlı yapısı olması seçim için önemlidir. RYU, temel OpenFlow desteği ile sınırlı olmamakla beraber OpenFlow protokolünün çeşitli sürümlerini (OpenFlow 1.0, 1.2, 1.3

vb.) desteklemektedir. Özellikle küçük ve orta ölçekli ağ mimarilerinin trafik planlaması ve kontrolü için uygun bir çözüm olarak görülen RYU framework'ü bu proje için seçilmiştir.

IV. LİTERATÜR TARAMASI VE TEORİK ALTYAPI

LEO uydu ağları, yüksek hareketlilik ve sık topoloji değişimi özelliklerini taşıdığından, bu ağların yönetiminde SDN mimarisi temel bir çözüm olarak sunulmaktadır. [2] SDN, kontrol düzlemini veri düzleminden ayırarak uyduların donanım karmaşıklığını azaltmaktadır ve uyduları sadece iletimden sorumlu "SDN ajanları" haline getirmektedir. Bu yapıda, OpenFlow protokolü kullanılarak merkezi bir kontrolcü (ör: Floodlight, Ryu) üzerinden akış tabloları, grup tabloları ve sayaçlar yönetilmektedir [3] [4]. Simülasyon aşamasında, Mininet gibi emülatörlerin tek bir makine üzerindeki CPU ve RAM kısıtlamaları nedeniyle, tam ölçekli 66 uydulu yapılar yerine daraltılmış topolojilerin kullanılması yaygın görülen bir pratiktir. [2] [5] Performans testlerinde, Java tabanlı Floodlight kontrolcüsünün, Python tabanlı Ryu kontrolcüsüne kıyasla daha yüksek bant genişliği ve daha düşük gecikme sağladığı tespit edilmiştir. Fakat Ryu, Python tabanlı olması sebebiyle hızlı prototipleme ve eğitim amaçlı çalışmalarda tercih edilmektedir. Trafik mühendisliği açısından, kontrolcünün "Link Discovery" ve "Topology Manager" modülleri ile entegre edilen Karınca Koloni Algoritması (ACO) gibi yöntemler, paket gecikmesi ve kuyruk uzunluklarında %15-22 oranında iyileşme sağlamaktadır. Ancak güvenlik perspektifinde, SDN mimarisi STRIDE analizlerine göre akış tablosu taşıması (DoS) ve zamanlama analizine dayalı bilgi ifşası tehditlerine açık olduğu tespit edilmiştir. Bu riskleri minimize etmek için LEO-SDN entegrasyonlarında hız sınırlama (rate limiting), olay filtreleme ve akış birleştirme (flow aggregation) gibi karşı önlemlerin uygulanması tavsiye edilmektedir. [3]

V. GELİŞTİRME / SİMÜLASYON ORTAMININ KURULUMU VE YAPILANDIRILMASI

A. Temel Platform Bileşenleri

- 1) Ubuntu 20.04 LTS
- 2) Python 3.8
- 3) Docker

B. Ağ Emülasyonu Bileşenleri

- 1) Mininet
- 2) Open vSwitch
- 3) iproute2

C. SDN Controller Bileşenleri

- 1) Ryu 4.34
- 2) eventlet 0.30.2
- 3) oslo.config 8.8.0
- 4) msgpack 1.0.4
- 5) ovs 2.17.0
- 6) netaddr 0.8.0
- 7) tinypc 1.1.4

D. Test ve Saldırı Araçları

- 1) hping3 3.0.0
- 2) iperf3 3.9
- 3) tcpdump 4.99.1
- 4) ping iputils

E. Topoloji Özellikleri

```
# configs/config.yaml
topology:
  satellites:
    count: 12
    orbital_planes: 3
    satellites_per_plane: 4
    altitude_km: 550
    inclination_deg: 53

  ground_stations:
    count: 4
    users_per_station: 3

  links:
    isl_delay_ms: 10
    isl_bandwidth_mbps: 1000
    downlink_delay_ms: 25
    downlink_bandwidth_mbps: 500
    user_delay_ms: 5
    user_bandwidth_mbps: 100
```

Listing 1: Simülasyon Konfigürasyonu (config.yaml)

TABLE IV
SİMÜLASYON TOPOLOJİ BİLEŞENLERİ

Bileşen	Sayı	Açıklama
OVS Switch	12	LEO uyduları (s1-s12)
Intra-plane ISL	12	Aynı düzlem içi bağlantılar (ring)
Inter-plane ISL	12	Düzlemler arası bağlantılar
Yer İstasyonu	4	Ground station host (gs1-gs4)
Kullanıcı Terminali	12	User terminal host (u1-u12)
Toplam Host	16	-
Toplam Link	40	-

F. Docker Image Oluşturma

Docker image oluşturma yaklaşık 15 dakika sürmektedir ve aşağıdaki katmanları içerir:

- 1) Ubuntu 20.04 base
- 2) Sistem paketleri (apt)
- 3) Mininet (source build)
- 4) Python paketleri (pip)
- 5) Proje dosyaları

VI. YAPILAN UYGULAMALAR VE TASARIM DETAYLARI

A. L2 Learning Switch Algoritması

SDN mimarisinde, özellikle OpenFlow protokolü kullanılarak uygulanan temel bir yönlendirme mantığı olarak bilinir. Geleneksel anahtarların donanım üzerinde yaptığı "MAC adresi öğrenme" işleminin, SDN'de merkezi kontrolcü tarafından yazılımsal olarak gerçekleştirilmesidir. Ağdaki bir switch paket aldığı anda şu adımlar gerçekleşmektedir:

- 1) Table-Miss: Anahtar, gelen paketin başlık bilgilerini (MAC adresini) kendi akış tablosunda (flow table) arar. Eğer eşleşen bir kural yoksa (table-miss), bu paket OpenFlow protokolünün Packet-In mesajı içine kapsüllenerek merkezi kontrolöre gönderilir. [1]
- 2) Learning: Kontrolcü (Ryu), gelen Packet-In mesajını inceler. Paketin kaynak MAC adresini ve hangi porttan (In-Port) geldiğini kaydederek, o MAC adresinin ağdaki konumunu öğrenir. [3] [6]
- 3) Flow-Mod: Kontrolcü, eğer hedef MAC adresinin yerini biliyorsa, anahtara bir Flow-Mod mesajı gönderir. Bu mesaj, "Şu hedef MAC adresine giden paketleri şu porta yönlendir" kuralını içerir. [3] [6]
- 4) Taşıma: Eğer kontrolcü hedef MAC adresinin yerini henüz bilmiyorsa, paketin geldiği port hariç diğer tüm portlara gönderilmesi (flood) emrini verir. [1]

Algorithm 1 Paket İşleme Fonksiyonu (Packet In)

```

0: function HANDLE_PACKET_IN(event)
0:   dpid ← event.datapath.id
0:   in_port ← event.match['in_port']
0:   eth_src ← parse_ethernet(event.data).src
0:   eth_dst ← parse_ethernet(event.data).dst
0:   {MAC tablosunu güncelle (öğrenme)}
0:   mac_table[dpid][eth_src] ← in_port
0:   {Hedef portu belirle}
0:   if eth_dst ∈ mac_table[dpid] then
0:     out_port ← mac_table[dpid][eth_dst]
0:   else
0:     out_port ← FLOOD
0:   end if
0:   {Flow kuralı ekle (flood değilse)}
0:   if out_port ≠ FLOOD then
0:     match ← {in_port, eth_src, eth_dst}
0:     actions ← [OUTPUT(out_port)]
0:     add_flow(priority
0:       500, match, actions, idle_timeout = 300)
0:   end if
0:   {Paketi ilet}
0:   send_packet_out(event.data, out_port)
0: end function=0

```

Yukarıdaki sözde kodda switch'e gelen her pakette kaynak MAC adresi ve giriş portu tabloya kaydedilmektedir (öğrenme). Hedef MAC tabloda varsa ilgili porta, yoksa tüm portlara (flood) gönderilmektedir. Bilinen hedefler için flow kuralı eklenerek sonraki paketler kontrolcüye gelmeden switch'te işlenmektedir.

B. Flow Manager - Akış Kuralı Ekleme

add_flow() fonksiyonu, SDN'in temelini oluşturmaktadır. Kontrolcünün switch'e "bu paketi şu porta gönder" demesini sağlamaktadır. Bu olmadan switch paketleri yönlendiremez. Saldırı tespitinde de bu fonksiyon ile saldırgan IP'si DROP kuralıyla engellenmektedir. Tüm ağ kontrolü bu fonksiyon üzerinden gerçekleşmektedir.

Algorithm 2 Akış Kuralı Ekleme Fonksiyonu (Add Flow)

```

0: function ADD_FLOW(priority, match, actions, idle, hard)
0:   {1. Aksiyonları instruction formatına dönüştür}
0:   instruction ← CREATE_INST(type =
0:     APPLY, actions)
0:   {2. FlowMod mesajı oluştur}
0:   flow_mod ← CREATE_FLOW_MOD(
0:     switch ← this.datapath,
0:     priority ← priority, {Kural önceliği}
0:     match ← match, {Eşleşme kriteri}
0:     instructions ← [instruction],
0:     idle_timeout ← idle, {Boşta silinme}
0:     hard_timeout ← hard {Mutlak silinme}
0:   )
0:   {3. Switch'e OpenFlow mesajı gönder}
0:   SEND_TO_SWITCH(flow_mod)
0: end function=0

```

C. LEO Topology - ISL Bağlantı Oluşturma

Bu yapı, LEO uydu konstellasyonunun gerçekçi mesh topolojisini oluşturmaktadır. Starlink benzeri ağlarda uydular hem kendi orbital düzlemindeki komşularla (intra-plane) hem de yan düzlemlerdeki uydularla (inter-plane) bağlantı kurmaktadır. Bu bağlantılar olmadan uydular arası veri iletimi mümkün değildir.

Algorithm 3 ISL Bağlantıları Oluşturma (Create ISL Links)

```

0: function CREATE_ISL_LINKS
0:   for all satellite ∈ satellites do
0:     plane ← satellite.plane_id
0:     pos ← satellite.position

0:     {=== INTRA-PLANE ISL (Ring) ===}
0:     {Aynı düzlemdeki sonraki uydu}
0:     next_pos ← (pos + 1) (mod sats_per_plane)
0:     next_sat ← GET_SAT(plane, next_pos)
0:     if next_sat ≠ NULL and sat.id < next.id then
0:       ADD_LINK(sat, next_sat, delay = 10ms)
0:     end if

0:     {=== INTER-PLANE ISL ===}
0:     {Komşu düzlemdeki aynı pozisyonundaki uydu}
0:     next_plane ← (plane + 1) (mod total_planes)
0:     adj_sat ← GET_SAT(next_plane, pos)
0:     if adj_sat ≠ NULL and sat.id < adj.id then
0:       ADD_LINK(sat, adj_sat, delay = 15ms)
0:     end if
0:   end for
0: end function=0

```

VII. İLK BULGULAR VE ÖN SONUÇLAR

Simülasyonun nasıl başlatıldığı gösterilmiş ve sonuçlar paylaşılmıştır.

```
# 1. Container Başlatma
docker run -it --privileged --name leo-sdn
↳ leo-sdn-attack

# 2. Controller Başlatma (Konteyner içinde)
ryu-manager sdn_controller/controller.py --verbose
↳ --ofp-tcp-listen-port 6633

# Topoloji başlatma (Konteyner içinde)
python3 topology/leo_topology.py
```

Listing 2: Simülasyon Başlatma Komutları

```
sahin@sahin-MacBook-Pro leo-sdn-attack % docker run -it
↳ --privileged --name leo-sdn leo-sdn-attack

=====
LEO SDN Attack Detection Environment
=====
OVS Status: ovs-vsctl (Open vSwitch) 2.13.8
Mininet:
Ryu: ryu-manager 4.34
=====

root@558769c3b1ba:~/leo-sdn-attack# ryu-manager
↳ sdn_controller/controller.py --verbose
↳ --ofp-tcp-listen-port 6633

loading app sdn_controller/controller.py
loading app ryu.controller.ofp_handler
instantiating app sdn_controller/controller.py of
↳ LEOController
=====
LEO SDN Controller Started
=====
instantiating app ryu.controller.ofp_handler of OFPHandler
BRICK LEOController
CONSUMES EventOFPPFlowStatsReply
CONSUMES EventOFPPacketIn
CONSUMES EventOFPPortStatsReply
CONSUMES EventOFPPStateChange
CONSUMES EventOFPSwitchFeatures
BRICK ofp_event
PROVIDES EventOFPPFlowStatsReply TO {'LEOController':
↳ {'main'}}
PROVIDES EventOFPPacketIn TO {'LEOController': {'main'}}
PROVIDES EventOFPPortStatsReply TO {'LEOController':
↳ {'main'}}
PROVIDES EventOFPPStateChange TO {'LEOController':
↳ {'main', 'dead'}}
PROVIDES EventOFPSwitchFeatures TO {'LEOController':
↳ {'config'}}
CONSUMES EventOFPEchoReply
CONSUMES EventOFPEchoRequest
CONSUMES EventOFPErrMsg
CONSUMES EventOFPHello
CONSUMES EventOFPPortDescStatsReply
CONSUMES EventOFPPortStatus
CONSUMES EventOFPSwitchFeatures
```

Listing 3: Docker Ortamında SDN Kontrolcünün Başlatılması

Gerçekleştirilen simülasyon testleri, LEO uydu ağ topolojisinin tasarlandığı gibi başarıyla oluşturulduğunu göstermektedir. Sistem, 3 orbital düzlemde toplam 12 uydu switch'i, 4 yer istasyonu ve 12 kullanıcı terminalini içeren karmaşık bir mesh topolojiyi Mininet ortamında simüle etmiştir. Inter-Satellite Link (ISL) bağlantıları, intra-plane (10ms) ve inter-plane (15ms) gecikme değerleriyle konfigüre edilmiştir. Tüm switch'ler OpenFlow 1.3 protokolü ile remote controller'a (127.0.0.1:6633) bağlanmış ve Spanning Tree Protocol (STP) aktifleştirilmiştir. Topoloji bilgileri (12 uydu, 4 yer istasyonu, 25ms downlink gecikmesi) konfigürasyon dosyasındaki parametrelerle tam uyumludur. Bu sonuçlar,

```
=====
LEO SDN ATTACK DETECTION - AĞ SİMÜLASYONU
=====

*** LEO Uydu Ağı Topolojisi Oluşturuluyor ***

+ Uydular oluşturuluyor...
- sat1-sat4 (Plane 0, Pos 0-3)
- sat5-sat8 (Plane 1, Pos 0-3)
- sat9-sat12 (Plane 2, Pos 0-3)

+ ISL bağlantıları oluşturuluyor...
- Intra-plane: sat1<->sat2, sat2<->sat3, ... [10ms]
- Inter-plane: sat1<->sat5, sat2<->sat6, ... [15ms]

+ Yer istasyonları oluşturuluyor...
- gs1 (10.0.0.1/24) -> sat1 [25ms]
- gs2 (10.0.0.2/24) -> sat2 [25ms]
- gs3 (10.0.0.3/24) -> sat3 [25ms]
- gs4 (10.0.0.4/24) -> sat4 [25ms]

+ Kullanıcılar oluşturuluyor...
- u1-u3 (10.0.0.5-7/24) -> sat1
- u4-u6 (10.0.0.8-10/24) -> sat2
- u7-u9 (10.0.0.11-13/24) -> sat3
- u10-u12 (10.0.0.14-16/24) -> sat4

*** Topoloji tamamlandı: 12 uydu, 4 yer istasyonu, 12
↳ kullanc ***
*** Remote Controller: 127.0.0.1:6633
*** Creating network
*** Adding hosts: gs1 gs2 gs3 gs4 u1-u12
*** Adding switches: s1-s12
*** Adding links: (gs1,s1) (gs2,s2) ... (s1,s2) (s1,s5) ...
*** Starting 12 switches
*** STP enabled on all switches ***
*** Ağ başlatıldı ***

-----
TOPOLOJİ BİLGİLERİ:
-----
satellites: 12
orbital_planes: 3
satellites_per_plane: 4
ground_stations: 4
users: 12
isl_delay_ms: 10
downlink_delay_ms: 25
-----

*** Starting CLI:
mininet>
```

Listing 4: Ağ Simülasyonu Başlatma Logları

SDN tabanlı LEO ağ simülasyonunun temel altyapısının çalışır durumda olduğunu doğrulamaktadır.

VIII. PROJE PLANI İLE KARŞILAŞTIRMA VE MEVCUT DURUM

Docker ortam kurulumu, LEO topoloji simülasyonu ve SDN kontrolcü geliştirme adımları zamanında tamamlanmıştır. Ancak, Ryu framework'ünün Python versiyon uyumsuzlukları ve eventlet bağımlılık sorunları nedeniyle kurulum aşamasında yaklaşık 1 haftalık gecikme yaşanmıştır. Bu sorun, Ubuntu 20.04 ve eventlet 0.30.2 sürümüne geçilerek çözülmüştür. Gecikme yaşanmasının bir diğer nedeni ise çalıştığım kurumda 3 hafta boyunca mesaiye kalmamdır. Mevcut durumda, saldırı simülasyonu (TCP SYN/ACK Flood) ve saldırı tespit/önleme modülleri henüz tamamlanmamıştır. Bu bileşenler, kalan süre içinde öncelikli olarak geliştirilecektir. Temel altyapının %70 oranında tamamlanmış olması, projenin başarıyla sonuçlandırılacağını göstermektedir.

IX. KARŞILAŞILAN GÜÇLÜKLER VE GELİŞTİRİLEN ÇÖZÜMLER

Proje sürecinde çeşitli teknik zorluklarla karşılaşmıştır. İlk olarak, Ubuntu 22.04 (Python 3.10) üzerinde Ryu framework'ü çalıştırılırken `TypeError: cannot set 'is_timeout' attribute of immutable type 'TimeoutError'` hatası alınmıştır. eventlet kütüphanesinin Python 3.10 ile uyumsuzluğundan kaynaklanan bu sorun, Ubuntu 20.04 (Python 3.8) ve eventlet 0.30.2 sürümüne geçilerek çözülmüştür. İkinci olarak, apt ile kurulan Mininet paketi Python 2.7 modüllerini içerdiğinden, Mininet GitHub'dan kaynak kodla derlenerek Python 3 uyumluluğu sağlanmıştır. Üçüncü olarak, Docker container içinde OVS kernel modülü yüklenemediğinden (modprobe: FATAL: Module openvswitch not found), OVS userspace modunda çalıştırılmıştır. Son olarak, mesh topolojideki broadcast storm sorunu, tüm switch'lerde STP aktifleştirilerek çözülmüştür. Bu deneyimler, sanallaştırma ortamlarında SDN geliştirmenin bağımlılık yönetimi ve uyumluluk açısından dikkatli planlama gerektirdiğini göstermiştir.

X. PROJENİN KALAN KISMI İÇİN REVİZE EDİLMİŞ ÇALIŞMA PLANI

A. Saldırı Simülasyonu

TCP SYN Flood ve TCP ACK Flood saldırı scriptleri geliştirilecektir. hping3 aracı kullanılarak 1000-10000 paket/saniye aralığında configurable saldırı simülasyonu gerçekleştirilecektir. hping3, özelleştirilmiş TCP/UDP/ICMP paketleri gönderebilen bir ağ aracıdır. Güvenlik testleri ve DDoS simülasyonu için kullanılmaktadır.

B. Saldırı Tespiti

Üç aşamalı hibrit tespit sistemi geliştirilecektir. İlk olarak eşik tabanlı tespit (SYN paket sayısı $>1000/s$), ardından entropy tabanlı anomali analizi ve son olarak makine öğrenmesi tabanlı sınıflandırma implementasyonu yapılacaktır. Flow istatistiklerinden feature extraction (pps, bps, flow count, SYN ratio) gerçekleştirilecek ve model sklearn kütüphanesi ile eğitilecektir.

C. Saldırı Önleme

Tespit edilen saldırganlara karşı otomatik müdahale mekanizması geliştirilecektir. FlowManager modülü üzerinden IP engelleme, OpenFlow meter kuralları ile rate limiting ve timeout tabanlı engel kaldırma işlevleri eklenecektir.

REFERENCES

- [1] "OpenFlow switch specification version 1.3.1 (wire protocol 0x04)," specification, Open Networking Foundation, September 2012.
- [2] M. Balta and Özçelik, "Şehir içi kavşak yönetim sistemleri için SDN temelli bir VANET mimari önerisi (a proposal of SDN based VANET architecture for urban intersection management systems)," *Journal of the Faculty of Engineering and Architecture of Gazi University*, vol. 34, no. 3, pp. 1451–1468, 2019.
- [3] R. Klöti, V. Kotronis, and P. Smith, "OpenFlow: A security analysis," in *2013 IEEE 21st International Conference on Network Protocols (ICNP)*, IEEE, 2013.
- [4] Y. Li, X. Guo, X. Pang, B. Peng, X. Li, and P. Zhang, "Performance analysis of Floodlight and Ryu SDN controllers under Mininet simulator," in *Proceedings of the IEEE*, 2020. Major Scientific and Technological Projects of CNPC under Grant ZD2019-183-006.

[5] Mininet, *Mininet Sample Workflow*. Documentation excerpts.

[6] S. R. Chowdhury, "Software defined networking tutorial." University of Waterloo, CS856 Course Material, 2015. Fall 2015.